SOATV HDMI Display and Playback Debugging

Investigation record and implementation guide | Raspberry Pi 4, Buildroot, Flutter eLinux GBM/DRM Record through 31 October 2026

Executive summary

Playback judder and unexpected HDMI modes were traced to two layers. The Pi boot arguments held the kernel and psplash at 1920x1080 at 60 Hz, but Flutter's eLinux GBM/DRM backend performed another modeset after launch. It selected connector->modes[0], which could be the TV's preferred 4096x2160 at 24 Hz or another 4K mode. The source was changed to prefer 1920x1080 at 60 Hz. GBM already uses the selected drm_mode_info_ for both the CRTC modeset and GBM surface dimensions.

The modified library built successfully as AArch64. SOATV initially failed to start because the new library depended on libuv.so.1, which the Buildroot image did not contain. Copying libuv from the Azure build instance restored startup, as reported. The active DRM mode and smoothness after that final deployment still need to be confirmed on the Pi.

System and symptoms

The target is a Raspberry Pi 4 with a minimal Buildroot image and a Flutter eLinux SOATV player connected to a TCL 4K television. Symptoms included playback judder, unexpected 4K output modes, intermittent blank/no-signal behavior during boot or HDMI hotplug, and an ALSA busy/fallback issue during pipeline rebuilds.

Playback evidence supplied

Measurement	Observation	Meaning
Stream	1280x720 at 30 FPS	Decoded source frames; independent of HDMI output mode
Decoder	Steady 30 FPS	Decoder kept up in the reported sample
Buffer	46-60 seconds	Buffering was healthy in that capture
Network	Generally 9-24 Mbps	No clear persistent bandwidth shortage in supplied logs
Flutter UI work	About 1-3 ms	UI-thread work was small in the sample
Raster timings	Repeated 41.6, 83.2 and 208 ms	Rendering missed multiple refresh deadlines; 4K raster load is a plausible contributor
Surface / DRM plane	4096x2160 mentioned initially; later DRM state showed 3840x2160	Observations differed across runs; active DRM state is authoritative

A 60 Hz refresh interval is about 16.67 ms. Raster work of 41.6 ms or longer misses multiple deadlines. Therefore, 3840x2160 at 60 Hz can still judder if Flutter cannot render the 4K surface in time.

Memory evidence

Several free -h snapshots showed free RAM falling from about 662 MB to 555 MB and then 511 MB. Available memory stayed around 1.1-1.2 GB; a later snapshot recovered to about 857 MB free as buff/cache decreased. This alone did not prove a Flutter leak or system-wide exhaustion. On Linux, MemAvailable is more useful than MemFree because reclaimable cache can be released. The root filesystem was 1.5 GB with about 808 MB used (60%); /appdata was separate. Swap was disabled.

Stream resolution versus HDMI mode

A 1280x720 stream remains a 720p decoded frame when the Pi sends a 3840x2160 HDMI signal. The rendering path scales it to the output surface. The television receives the HDMI mode and normally does not choose that mode based on the stream.

For 30 FPS content, 60 Hz gives a clean two-refresh cadence. 4096x2160 at 24 Hz cannot present 30 unique frames evenly. But 3840x2160 at 60 Hz only fixes output cadence; it does not guarantee smooth rendering if raster work exceeds 16.67 ms.

Mode	Meaning in this investigation
1280x720 at 30 FPS	Stream/decoded frame size and rate
1920x1080 at 60 Hz	Initial kernel and psplash HDMI output
4096x2160 at 24 Hz	Seen on TV OSD in an earlier observation
3840x2160 at 60 Hz	Later active DRM mode while Flutter ran

The 4096x2160 at 24 Hz mode was not a Flutter resource ceiling. Later DRM state showed Flutter using 3840x2160 at 60 Hz. Rendering capacity affects missed frames, not normally which modes DRM enumerates.

Debugging chronology and evidence

1. Playback logs first pointed to display cadence and rendering load: a 4K Flutter surface, 720p30 stream, steady decoder output, healthy buffering, low UI-thread work and high raster durations.
2. Buildroot was minimal: vmstat was unavailable and BusyBox dmesg did not support dmesg -w. /proc/meminfo, /proc/<pid>/status, /proc/loadavg, top, sysfs DRM state and debugfs were used instead.
3. U-Boot bootargs were changed to include video=HDMI-A-1:1920x1080@60D. /proc/cmdline later confirmed it. psplash stayed at 1080p60, so the kernel-side setting worked.
4. Flutter changed the mode after launch. /sys/kernel/debug/dri/1/state showed mode: "3840x2160": 60 and a plane allocated by io.flutter.rast with size=3840x2160.
5. The deployed library was libflutter_elinux_gbm.so. It imported drmModeGetConnector, drmModeSetCrtc and gbm_surface_create, identifying the GBM/DRM path.
6. Source was traced to Sony flutter-embedded-linux. ConfigureDisplay() in native_window_drm.cc assigned drm_mode_info_ = connector->modes[0]. The GBM backend used drm_mode_info_ for drmModeSetCrtc() and gbm_surface_create().
7. HDMI hotplug was tested. Both connectors sometimes reported disconnected, zero-byte EDID and no modes. After moving the cable to HDMI-A-2, sysfs reported connected while the screen remained blank. Killing Flutter let inittab respawn it; the fresh process initialized successfully. In the observed run, the backend did not recover cleanly when HDMI became ready after Flutter had started.
8. Audio was clarified as separate: audio worked at initial launch, then HDMI ALSA could be busy during pipeline teardown/rebuild and player fell back to plughw:0,0. This was not initial HDMI detection failure.

Minimal Buildroot diagnostics

The sysfs modes file lists supported modes, not necessarily the active mode. The active DRM state was decisive.

```
cat /proc/cmdline

for p in /sys/class/drm/card*-HDMI-A-*; do
  echo "=== $p ==="
  cat "$p/status" 2>/dev/null
  wc -c < "$p/edid" 2>/dev/null
  cat "$p/modes" 2>/dev/null
done

mount -t debugfs debugfs /sys/kernel/debug 2>/dev/null || true
grep -E -i 'connector|crtc|active|mode' /sys/kernel/debug/dri/1/state
grep -E -A15 -B5 'io.flutter.rast' /sys/kernel/debug/dri/1/state
```

modetest was absent. A diagnostic Buildroot image can include it with BR2_PACKAGE_LIBDRM_INSTALL_TESTS=y.

Source and artifact trace

The SOATV bundle linked libflutter_elinux_gbm.so. The app's elinux/flutter/CMakeLists.txt selected it from generated ephemeral output. Flutter eLinux tools at /opt/flutter-elinux manage and download prebuilt engine artifacts; they are not the DRM implementation source.

The SDK engine.version was cf56914b326edb0ccb123ffdc60f00060bd513fa. Its artifact cache points to Sony flutter-embedded-linux releases and supports ELINUX_ENGINE_BASE_URL and ELINUX_ENGINE_BASE_LOCAL_DIRECTORY overrides. The source tag list exposed cf56914b32; the GitHub release label included engine-artifacts-. The fork did not automatically include upstream tags, so upstream tags were fetched and branch feat/drm-mode-policy was created from cf56914b32.

```
git remote add upstream https://github.com/sony/flutter-embedded-linux.git
git fetch upstream --tags
git switch -c feat/drm-mode-policy cf56914b32
```

Source changed:

```
src/flutter/shell/platform/linux_embedded/window/native_window_drm.cc
```

Only the embedder repository needed a code change for mode selection. flutter-elinux can remain unchanged when custom artifacts are supplied through its supported overrides.

DRM mode-selection change

The old ConfigureDisplay() code selected connector->modes[0]. The replacement policy:

1. Prefers 1920x1080 at about 60 Hz as a conservative Raspberry Pi default.
2. If unavailable, chooses the 59-60 Hz mode with the smallest pixel count.
3. If no 60 Hz mode exists, chooses the smallest mode in the 50-60 Hz range.
4. If none qualifies, retains the first-mode fallback.

The code checks connector->count_modes, assigns the selected drmModeModeInfo to drm_mode_info_, preserves rotation handling, and logs mode name, dimensions and refresh. The GBM source already uses drm_mode_info_ for both CRTC modeset and GBM surface size, so the selected resolution drives both.

This is a stable output policy chosen at Flutter display initialization; it does not switch refresh for each stream. Flutter logical UI sizing and device-pixel ratio are separate from HDMI mode; check those independently if icons appear too small.

Build process

The build machine reported uname -m as aarch64, so it built natively for the Pi CPU architecture. CMake used Clang 18.1.3 and found EGL, xkbcommon, libdrm 2.4.125, libinput 1.25.0, libudev 255, GBM 25.2.8 and libuv 1.48.0. libsystemd was not found, so the build selected libuv.

```
cmake -S . -B build-soatv-arm64-release -DBUILD_ELINUX_SO=ON -DBACKEND_TYPE=DRM-GBM -
DFLUTTER_RELEASE=ON -DCMAKE_BUILD_TYPE=Release
cmake --build build-soatv-arm64-release --parallel
```

Compilation reached native_window_drm.cc, but the first link attempt failed because CMake expected:

```
/home/azureuser/flutter-embedded-linux/build/libflutter_engine.so
```

The matching engine was found at:

```
/opt/flutter-elinux/flutter/bin/cache/artifacts/engine/elinux-arm64-release/libflutter_engine.so
```

It was copied to the expected source build path. The build then produced:

```
build-soatv-arm64-release/libflutter_elinux_gbm.so
```

The file command identified an ELF 64-bit LSB shared object for ARM AArch64. This confirms architecture, but not compatibility with every library in the Buildroot image.

Runtime dependency failure and recovery

The deployed SOATV executable initially failed before Flutter initialized:

```
libuv.so.1: cannot open shared object file: No such file or directory
```

The new embedder selected libuv because libsystemd was missing on the Azure build host. Buildroot did not contain libuv.so.1. The user copied libuv from Azure to the Pi and reported SOATV then started. The exact library build ID and long-term ABI compatibility were not recorded.

For a reproducible image, include libuv through Buildroot (for example BR2_PACKAGE_LIBUV=y if the selected version exposes it), or build/package a target-compatible libuv and dependencies. Do not assume an arbitrary host library is suitable for the target. Verify on the actual Buildroot filesystem.

```
find output/target -name 'libuv.so*' -print
find /lib /usr/lib /usr/local/lib -name 'libuv.so*' -print 2>/dev/null
ldd build-soatv-arm64-release/libflutter_elinux_gbm.so
```

Deployment and remaining validation

The modified library was built as AArch64 and the missing libuv dependency was resolved sufficiently for SOATV to start. Close the loop with these checks:

- Confirm deployed bundle contains rebuilt libflutter_elinux_gbm.so and matching libflutter_engine.so.
- Boot with TV connected and verify /proc/cmdline contains the intended video= parameter.
- While Flutter runs, inspect /sys/kernel/debug/dri/1/state. Confirm active mode is 1920x1080 at about 60 Hz and io.flutter.rast plane is 1920x1080.
- Repeat with HDMI-A-1 and HDMI-A-2 if both ports are supported. Test TV powered before Pi boot and powered/connected after boot.
- Play 720p30, 1080p and any 4K streams. Record decoder FPS, raster duration, dropped frames, audio continuity and active DRM mode.
- If 4K60 is considered, measure raster time against the 16.67 ms refresh budget. Correct HDMI cadence alone does not prove Flutter can render 4K smoothly.
- Package libuv through Buildroot or another controlled target-compatible method, then verify cold boot without manual library copying.
- Exercise channel reload and HDMI hotplug separately. The ALSA busy/fallback issue during pipeline rebuild remains separate unless independently fixed and verified.

Related audio finding

Audio worked at initial launch. During pipeline teardown/rebuild, HDMI ALSA could still be busy and the player sometimes fell back to plughw:0,0, routing sound to the Pi jack. The likely area is serialized pipeline teardown and reopening the same named HDMI ALSA device after the old sink releases it. This audio lifecycle fix was not part of the DRM mode-selection build.

Conclusions

- A 720p stream does not command the TV to output 720p. DRM mode describes the Pi HDMI signal and is independent of stream frame dimensions.
- The kernel video= argument controlled boot/psplash mode; Flutter later performed a userspace DRM modeset.

- The source-level cause was connector->modes[0] in the eLinux DRM backend.
- The 4096x2160 at 24 Hz observation was not a Flutter resource cap. Later DRM state showed 3840x2160 at 60 Hz; 4K rendering load can still cause missed frames.
- The source change built into an AArch64 GBM library. Final active mode and playback smoothness still need on-device measurement.
- Copying libuv.so.1 from Azure restored startup in the reported test. Buildroot packaging should make that dependency reproducible.

References

Sony Flutter eLinux tools: https://github.com/sony/flutter-elinux Sony Flutter embedded Linux source: https://github.com/sony/flutter-embedded-linux Sony embedder build guide: https://github.com/sony/flutter-embedded-linux/wiki/Building-Embedded-Linux-embedding-for-Flutter